Multivariate Optimization

From Local Geometry to Gradient Descent

Maroun Ayli, Youssef Bakouny

Saint Joseph University of Beirut

Reminder: Derivatives in One Dimension

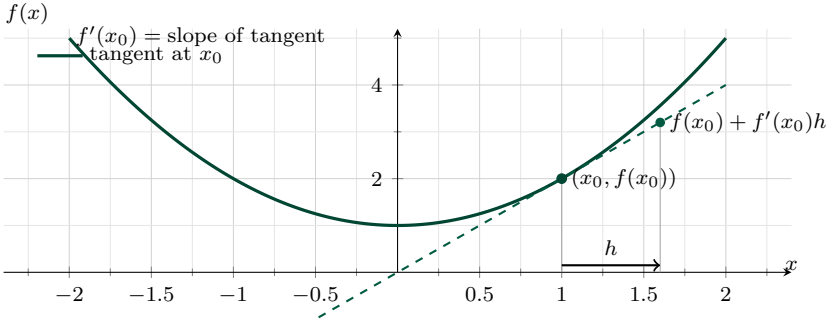
Let $f : \mathbb{R} \to \mathbb{R}$.

Derivative.

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}. \qquad \text{(slope of the tangent at } x\text{)}$$

Local linear approximation. For small h,

$$f(x+h) = f(x) + f'(x)\,h + o(h).$$



Reminder: Basic Differentiation Rules

Let $f, g : \mathbb{R} \to \mathbb{R}$ be differentiable functions and $c \in \mathbb{R}$.

Linearity.

$$(cf + g)'(x) = c\, f'(x) + g'(x)$$

Product rule.

$$(fg)'(x) = f'(x)\, g(x) + f(x)\, g'(x)$$

Quotient rule.

$$\left(\frac{f}{g}\right)'(x) = \frac{f'(x)g(x) - f(x)g'(x)}{g(x)^2}, \quad g(x) \neq 0$$

Chain rule. If $h(x) = f(g(x))$, then

$$h'(x) = f'(g(x))\, g'(x).$$

Common examples.

$$(x^n)' = nx^{n-1}, \quad (e^x)' = e^x, \quad (\log x)' = \frac{1}{x}.$$

Descent and Optimality in One Dimension

Let $f : \mathbb{R} \to \mathbb{R}$ be differentiable.

Sign of the derivative. For a point $x \in \mathbb{R}$,

$$\begin{aligned} f'(x) > 0 \;&\Rightarrow\; f \text{ is locally increasing,} \\ f'(x) < 0 \;&\Rightarrow\; f \text{ is locally decreasing.} \end{aligned}$$

Descent direction. A direction $d \in \mathbb{R}$ is a descent direction at x if

$$f'(x)\, d < 0.$$

In particular, choosing

$$d = -f'(x)$$

guarantees local decrease of f.

Necessary optimality condition. If $x^\star$ is a local minimum of f, then

$$f'(x^\star) = 0.$$

Move iteratively in the direction of *negative slope* to reduce the function value.

Second-Order Conditions in One Dimension

Let $f : \mathbb{R} \to \mathbb{R}$ be twice differentiable.

Stationary points. A point $x^\star$ is called *stationary* if

$$f'(x^\star) = 0.$$

Second-order test. Let $x^\star$ be a stationary point.

$$\begin{aligned} f''(x^\star) > 0 &\Rightarrow x^\star \text{ is a local minimum,} \\ f''(x^\star) < 0 &\Rightarrow x^\star \text{ is a local maximum,} \\ f''(x^\star) = 0 &\Rightarrow \text{test is inconclusive.} \end{aligned}$$

Interpretation. The second derivative measures *local curvature*: positive curvature corresponds to a locally convex shape.

Connection to optimization. First-order conditions identify candidates; second-order conditions classify them.

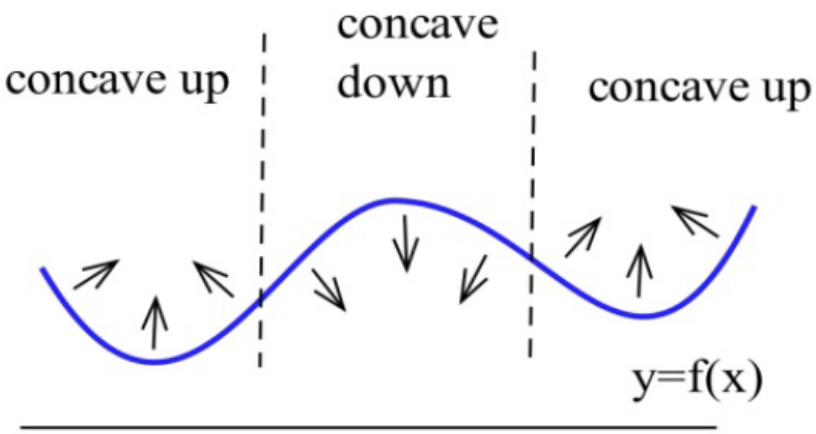


Figure: Second Order and Curvature

Reminder: Taylor Expansion in One Dimension

Let $f : \mathbb{R} \to \mathbb{R}$ be sufficiently smooth and let $x \in \mathbb{R}$.

First-order Taylor expansion. For small $h \in \mathbb{R}$,

$$f(x+h) = f(x) + f'(x)\,h + o(h).$$

Second-order Taylor expansion. If f is twice differentiable, then

$$f(x+h) = f(x) + f'(x)\,h + \tfrac{1}{2} f''(x)\,h^2 + o(h^2).$$

Optimization viewpoint.

- First-order term determines *descent*.
- Second-order term captures *curvature*.
- Local minima satisfy $f'(x) = 0$ and $f''(x) > 0$.

Optimization algorithms exploit local Taylor models to choose update directions.

Why First-Order Methods?

Consider minimizing $f : \mathbb{R} \to \mathbb{R}$.

Local Taylor models.

$$\text{First order:} \quad f(x+h) \approx f(x) + f'(x)\,h$$

$$\text{Second order:} \quad f(x+h) \approx f(x) + f'(x)\,h + \tfrac{1}{2} f''(x)\,h^2$$

Second-order methods (e.g. Newton).

- Require computing and storing $f''(x)$
- Sensitive to poor curvature estimates
- Expensive or unstable in large-scale problems

First-order methods.

- Use only $f'(x)$
- Cheap and robust per iteration
- Scale naturally to high dimensions

Key trade-off. Slower convergence per step, but vastly lower computational cost.

Iterative Optimization in One Dimension

Let $f : \mathbb{R} \to \mathbb{R}$ be differentiable.

Basic idea. Starting from an initial point x_0, generate a sequence

$$x_0, x_1, x_2, \ldots$$

that progressively reduces the function value.

First-order update rule. Using the local linear model,

$$x_{k+1} = x_k - \eta_k \, f'(x_k),$$

where $\eta_k > 0$ is the step size.

Interpretation.

- $f'(x_k)$ gives the local slope
- moving in direction $-f'(x_k)$ decreases f
- η_k controls the step length

Stopping criterion. Iterations stop when $|f'(x_k)|$ is sufficiently small.

Convergence Intuition in One Dimension

Consider the iterative scheme

$$x_{k+1} = x_k - \eta\, f'(x_k),$$

with constant step size $\eta > 0$.

Small step size (η too small).

- Guaranteed decrease of f
- Very slow progress
- Many iterations required

Large step size (η too large).

- Overshooting the minimum
- Oscillations around the optimum
- Possible divergence

Well-chosen step size.

- Balanced progress and stability
- Convergence to a stationary point

Why a Numerical Experiment?

So far, we derived an iterative method based on local slope information.

However:

- The update rule alone does not guarantee convergence
- The behavior depends strongly on the step size
- Different functions may exhibit very different dynamics

Goal of the experiment.

- Visualize the iterations of the algorithm
- Compare different choices of the step size
- Observe convergence, oscillations, or divergence

Local slope information guides the updates, but global behavior emerges only through iteration.

Experimental Setup (COLAB)

We consider the iterative scheme

$$x_{k+1} = x_k - \eta\, f'(x_k),$$

starting from an initial point $x_0 \in \mathbb{R}$.

Test functions.

- A convex function with a unique minimum
- A nonconvex function with multiple stationary points

Parameters.

- Same initialization x_0
- Different constant step sizes $\eta > 0$
- Fixed number of iterations

Visualization. At each iteration, we plot the point $(x_k, f(x_k))$ on the graph of the function.

Choosing the Step Size: Practical Strategies

The step size η plays a central role in iterative optimization.

Fixed step size.

- Simple and inexpensive
- Requires careful manual tuning
- Sensitive to the scale of the function

Line search methods.

- Choose η at each iteration to ensure sufficient decrease
- Adapt to the local shape of the function
- More reliable, but more computationally expensive

Adaptive step sizes.

- Automatically adjust η during iterations
- Reduce sensitivity to initialization
- Common in large-scale and learning-based settings

Multivariate Scalar Function Optimization

From one-dimensional intuition to high-dimensional methods

We now consider real-valued functions of several variables,

$$f : \mathbb{R}^n \to \mathbb{R}, \qquad x = (x_1, \ldots, x_n).$$

In contrast to the one-dimensional case, there is no single notion of slope: the function may increase in some directions and decrease in others.

Local variation is described through derivatives with respect to each coordinate direction.

Our goal is to extend the one-dimensional ideas by identifying a direction that summarizes first-order behavior in all coordinates.

This requires assembling partial derivative information into a single object.

Partial Derivatives

Let $f : \mathbb{R}^n \to \mathbb{R}$ and let $x = (x_1, \ldots, x_n) \in \mathbb{R}^n$.

The partial derivative of f with respect to the i-th variable at x is defined by

$$\frac{\partial f}{\partial x_i}(x) = \lim_{h \to 0} \frac{f(x_1, \ldots, x_i + h, \ldots, x_n) - f(x_1, \ldots, x_i, \ldots, x_n)}{h},$$

whenever the limit exists.

Each partial derivative measures the local rate of change of f along a single coordinate direction, while keeping all other variables fixed.

For a point x, the collection

$$\left(\frac{\partial f}{\partial x_1}(x), \ldots, \frac{\partial f}{\partial x_n}(x) \right)$$

encodes first-order information in all coordinate directions.

The Gradient

Let $f : \mathbb{R}^n \to \mathbb{R}$ be differentiable and $x \in \mathbb{R}^n$.

The gradient of f at x is the vector of partial derivatives

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(x) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x) \end{pmatrix} \in \mathbb{R}^n.$$

The gradient collects all first-order variation of f at x into a single vector-valued object.

It provides a linear approximation of f around x:

$$f(x+h) = f(x) + \nabla f(x)^\top h + o(\|h\|), \qquad h \in \mathbb{R}^n.$$

This expression generalizes the one-dimensional Taylor expansion.

Directional Derivatives

Let $f : \mathbb{R}^d \to \mathbb{R}$ be differentiable and $x \in \mathbb{R}^d$.

For a direction $v \in \mathbb{R}^d$, the directional derivative of f at x along v is defined as

$$D_v f(x) = \lim_{h \to 0} \frac{f(x + hv) - f(x)}{h},$$

whenever the limit exists.

If f is differentiable, then

$$D_v f(x) = \nabla f(x)^\top v.$$

Among all unit directions $\|v\| = 1$, the directional derivative is maximized when

$$v = \frac{\nabla f(x)}{\|\nabla f(x)\|}.$$

This identifies the gradient as the direction of steepest local increase.

Partial vs. Directional Derivatives

Partial derivative (along a coordinate axis). For $f : \mathbb{R}^n \to \mathbb{R}$ and $x = (x_1, \ldots, x_n)$,

$$\frac{\partial f}{\partial x_i}(x) = \lim_{h \to 0} \frac{f(x_1, \ldots, x_i + h, \ldots, x_n) - f(x_1, \ldots, x_i, \ldots, x_n)}{h}.$$

Interpretation: vary only x_i and keep all other coordinates fixed.

Directional derivative (along any direction). For a direction $v \in \mathbb{R}^n$,

$$D_v f(x) = \lim_{h \to 0} \frac{f(x + hv) - f(x)}{h}.$$

Interpretation: move from x along the line $x + hv$.

Key identity

Let $e_i = (0, \ldots, 0, 1, 0, \ldots, 0)$ be the i-th standard basis vector. Then

$$\frac{\partial f}{\partial x_i}(x) = D_{e_i} f(x).$$

Important: Partial derivatives are directional derivatives *along the coordinate axes.*

Steepest Direction of Variation

Consider the directional derivative

$$D_v f(x) = \nabla f(x)^\top v, \qquad v \in \mathbb{R}^d.$$

To compare directions independently of scale, restrict to

$$\|v\| = 1.$$

By the Cauchy–Schwarz inequality,

$$\nabla f(x)^\top v \leq \|\nabla f(x)\|,$$

with equality if and only if

$$v = \frac{\nabla f(x)}{\|\nabla f(x)\|}.$$

This direction yields the maximal local rate of increase of f.

Steepest Descent Direction

Let $f : \mathbb{R}^d \to \mathbb{R}$ be differentiable and $x \in \mathbb{R}^d$.

For a unit direction $v \in \mathbb{R}^d$, the rate of change of f at x along v is given by

$$\nabla f(x)^\top v.$$

This quantity is minimized when

$$v = -\frac{\nabla f(x)}{\|\nabla f(x)\|}.$$

This direction yields the largest local decrease of f among all directions of unit length.

It provides a natural generalization of the negative slope from the one-dimensional case.

First-Order Optimality Condition

Let $f : \mathbb{R}^d \to \mathbb{R}$ be differentiable and $x^\star \in \mathbb{R}^d$.

If $x^\star$ is a local minimum of f, then

$$\nabla f(x^\star) = 0.$$

Indeed, for any direction $v \in \mathbb{R}^d$, the directional derivative at $x^\star$ must satisfy

$$\nabla f(x^\star)^\top v = 0.$$

Such points are called *stationary points*. They include local minima, local maxima, and saddle points.

Local minima, Local maxima, and Saddle Points

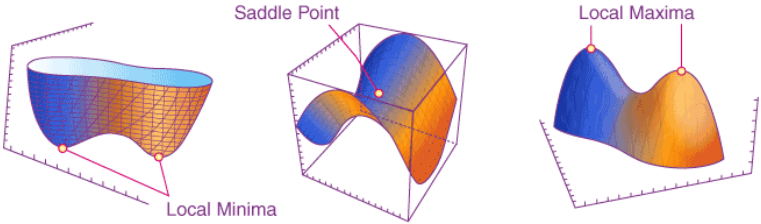


Figure: Local minima, local maxima, and saddle points

Gradient Descent

Let $f : \mathbb{R}^d \to \mathbb{R}$ be differentiable.

Starting from an initialization $x_0 \in \mathbb{R}^d$, define the iterates

$$x_{k+1} = x_k - \eta_k \nabla f(x_k), \qquad k = 0, 1, 2, \ldots$$

where $\eta_k > 0$ is a step size.

For a fixed step size $\eta_k = \eta$,

$$x_{k+1} = x_k - \eta \nabla f(x_k).$$

A typical stopping condition is

$$\|\nabla f(x_k)\| \leq \varepsilon \quad \text{or} \quad k \geq k_{\max}.$$

Practical Behavior of Gradient Descent

The update

$$x_{k+1} = x_k - \eta \nabla f(x_k)$$

is simple, but its behavior can vary significantly in practice.

Convergence depends on:

- the choice of step size η
- the local geometry of the objective function
- the scaling of different coordinates

Even for smooth functions, the sequence (x_k) may:

- converge slowly,
- oscillate,
- or stagnate far from an optimum.

These effects become more pronounced in high-dimensional settings.

Limitations of Gradient Descent

Several structural issues limit the efficiency of basic gradient descent.

If the level sets of f are highly elongated, the gradient may point toward steep directions rather than toward the minimum, leading to slow progress.

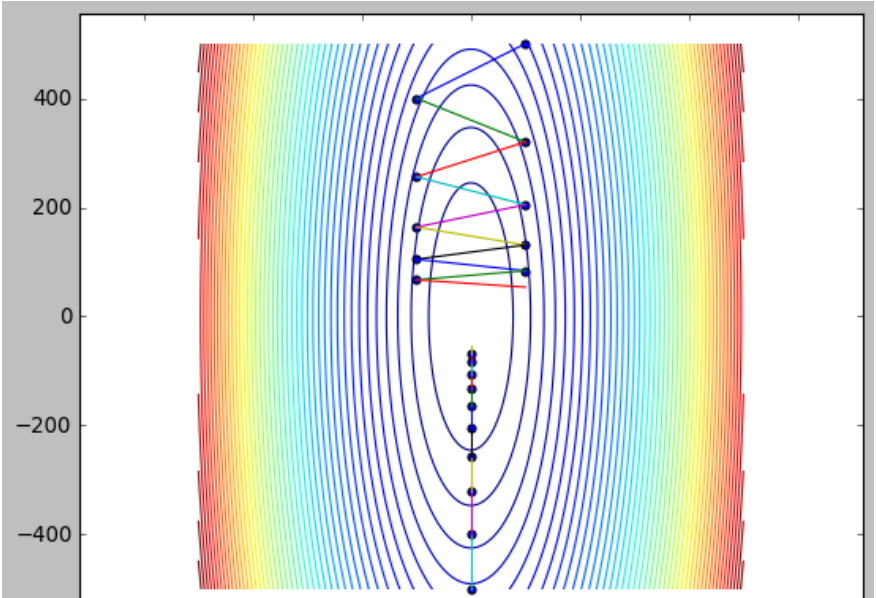
The same step size is applied to all coordinates, even if the function varies at very different scales along different directions.

In nonconvex problems:

- convergence to saddle points is possible,
- different initializations may lead to different solutions.

These limitations motivate modified update rules and improved step-size strategies.

Gradient Descent Zig Zag



Momentum

Basic gradient descent updates the parameters using only the current gradient.

Momentum introduces a running velocity:

$$
\begin{aligned}
v_{k+1} &= \beta v_k + \nabla f(x_k), \\
x_{k+1} &= x_k - \eta v_{k+1},
\end{aligned}
$$

with $\beta \in (0, 1)$.

The update accumulates gradient information over time, leading to smoother trajectories.

This reduces oscillations and accelerates progress along directions of consistent descent.

Scaling and Preconditioning

Gradient descent applies the same step size to all coordinates:

$$x_{k+1} = x_k - \eta \nabla f(x_k).$$

When different coordinates have different scales, this can lead to inefficient zig-zagging behavior.

A more general update takes the form

$$x_{k+1} = x_k - \eta \, P_k \nabla f(x_k),$$

where P_k is a positive definite matrix.

Such transformations rescale directions of descent to better match the geometry of the objective.

Adaptive Step Sizes

In many problems, different coordinates evolve at different rates.

Adaptive methods adjust the step size during optimization, often on a per-coordinate basis.

Typical strategies:

- decrease the step size over time,
- normalize updates using past gradient magnitudes,
- combine scaling with momentum.

These approaches reduce sensitivity to parameter tuning and improve robustness in large-scale problems.

Step Size Selection: Fixed and Diminishing

In gradient-based methods, the update has the form

$$x_{k+1} = x_k - \eta_k \nabla f(x_k).$$

A *fixed step size* uses

$$\eta_k = \eta > 0.$$

This choice:

- is simple and computationally cheap,
- often works well in practice when properly tuned,
- may lead to divergence or oscillations if η is too large.

A *diminishing step size* satisfies

$$\eta_k \downarrow 0, \qquad \sum_{k=1}^{\infty} \eta_k = \infty.$$

Such sequences are common in convergence analysis, but may result in slow progress in practice.

Limitations of Full Gradient Computation

So far, we assumed access to the full gradient $\nabla f(x)$.

In many applications, the objective has the form

$$f(x) = \frac{1}{N} \sum_{i=1}^{N} f_i(x),$$

where N may be very large.

Computing the full gradient requires

$$\nabla f(x) = \frac{1}{N} \sum_{i=1}^{N} \nabla f_i(x),$$

which scales linearly with N.

For large datasets, this makes each iteration expensive, even when gradient descent converges in few iterations.

Approximating the Gradient

Instead of using the full gradient, one can approximate it using a small subset of terms.

Let $i_k \in \{1, \ldots, N\}$ be a randomly selected index. An unbiased approximation of the gradient is

$$g_k = \nabla f_{i_k}(x_k), \qquad \mathbb{E}[g_k] = \nabla f(x_k).$$

This approximation is much cheaper to compute, but introduces randomness into the updates.

The optimization method trades exact descent for faster, noisier iterations.

Stochastic Gradient Approximation

At iteration k, select a random index

$$i_k \in \{1, \ldots, N\}$$

according to a specified sampling rule (typically uniform).

Define the stochastic gradient

$$g_k = \nabla f_{i_k}(x_k).$$

This estimator satisfies:

$$\mathbb{E}[g_k \mid x_k] = \nabla f(x_k),$$

but

$$\mathbb{E}\left[\|g_k - \nabla f(x_k)\|^2\right] > 0.$$

Thus, stochastic gradients are:

- unbiased,
- noisy,
- significantly cheaper to compute.

Stochastic Gradient Descent Algorithm

Given an initial point $x_0 \in \mathbb{R}^d$, stochastic gradient descent iterates

$$x_{k+1} = x_k - \eta_k g_k, \qquad g_k = \nabla f_{i_k}(x_k).$$

Compared to gradient descent:

- each iteration is much cheaper,
- updates are no longer deterministic,
- the objective value may not decrease at every step.

The behavior of SGD depends critically on:

- the step size sequence (η_k),
- the variance of the stochastic gradients,
- the structure of the objective function.

Gradient of MSE for Linear Regression

We fit a linear model

$$\hat{y}_i = w\,x_i + b$$

to data (x_i, y_i) by minimizing the **mean squared error** (MSE):

$$L(w,b) = \frac{1}{n}\sum_{i=1}^{n}\big(w\,x_i + b - y_i\big)^2.$$

Goal: compute the gradient

$$\nabla L(w,b) = \left(\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}\right).$$